

Vulkan Notes

Depth Buffering

Adam HAMOUC

June 2026

1. Principe

Sans depth buffer, les objets sont rendus dans l'ordre du tableau de vertices — le dernier dessiné passe par-dessus tout. Le depth buffer (ou Z-buffer) resout ce probleme : pour chaque pixel, on stocke la profondeur du fragment le plus proche. Si un nouveau fragment est plus loin, il est rejete.

Convention Vulkan : 0.0 = plan near, 1.0 = plan far. La valeur de clear initiale doit donc etre 1.0 pour que tous les fragments puissent s'afficher.

Le depth buffer est un attachement supplementaire du framebuffer, au meme titre que l'attachement couleur. Un seul depth buffer suffit pour toutes les images de la swap chain car les subpasses sont sequentielles.

2. Format et creation des ressources

Le depth buffer necessite une `VkImage`, de la memoire et une `VkImageView` — comme toute image Vulkan.

Formats disponibles (par ordre de preference) :

Format	Description
<code>VK_FORMAT_D32_SFLOAT</code>	32 bits depth seul
<code>VK_FORMAT_D32_SFLOAT_S8_UINT</code>	32 bits depth + 8 bits stencil
<code>VK_FORMAT_D24_UNORM_S8_UINT</code>	24 bits depth + 8 bits stencil

Le format optimal se choisit avec `vkGetPhysicalDeviceFormatProperties` — tous les GPU ne supportent pas tous les formats.

```
VkFormat findDepthFormat() {
    return findSupportedFormat(
        {VK_FORMAT_D32_SFLOAT,
         VK_FORMAT_D32_SFLOAT_S8_UINT,
         VK_FORMAT_D24_UNORM_S8_UINT},
        VK_IMAGE_TILING_OPTIMAL,
        VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT);
}

void createDepthResources() {
    VkFormat depthFormat = findDepthFormat();
    createImage(swapChainExtent.width, swapChainExtent.height,
        depthFormat, VK_IMAGE_TILING_OPTIMAL,
        VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT,
        VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT,
        depthImage, depthImageMemory);
    depthImageView = createImageView(depthImage, depthFormat,
        VK_IMAGE_ASPECT_DEPTH_BIT);
}
```

`createImageView` doit accepter un parametre `aspectFlags` pour pouvoir specifier `DEPTH_BIT` au lieu de `COLOR_BIT`.

3. Modifications de la Render Pass

```
VkAttachmentDescription depthAttachment{};
depthAttachment.format      = findDepthFormat();
depthAttachment.samples     = VK_SAMPLE_COUNT_1_BIT;
depthAttachment.loadOp      = VK_ATTACHMENT_LOAD_OP_CLEAR;
depthAttachment.storeOp     = VK_ATTACHMENT_STORE_OP_DONT_CARE; // pas besoin apres le
    rendu
depthAttachment.stencilLoadOp = VK_ATTACHMENT_LOAD_OP_DONT_CARE;
depthAttachment.stencilStoreOp = VK_ATTACHMENT_STORE_OP_DONT_CARE;
depthAttachment.initialLayout = VK_IMAGE_LAYOUT_UNDEFINED;
depthAttachment.finalLayout   = VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL;

VkAttachmentReference depthRef{};
depthRef.attachment = 1; // index dans le tableau des attachements
depthRef.layout     = VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL;

subpass.pDepthStencilAttachment = &depthRef; // une seule ref depth par subpass

// Passer les deux attachements a la render pass
std::array<VkAttachmentDescription, 2> attachments = {colorAttachment, depthAttachment};
renderPassInfo.attachmentCount = 2;
renderPassInfo.pAttachments    = attachments.data();
```

4. Framebuffer, Clear Values et Pipeline

Framebuffer — ajouter la depth image view :

```
std::array<VkImageView, 2> attachments = {swapChainImageViews[i], depthImageView};
framebufferInfo.attachmentCount = 2;
framebufferInfo.pAttachments    = attachments.data();
```

Clear values — une par attachement, dans le meme ordre :

```
std::array<VkClearColor, 2> clearValues{};
clearValues[0].color        = {{0.0f, 0.0f, 0.0f, 1.0f}};
clearValues[1].depthStencil = {1.0f, 0}; // depth=1.0 (far), stencil=0
renderPassInfo.clearValueCount = 2;
renderPassInfo.pClearValues    = clearValues.data();
```

Pipeline — activer le depth test dans createGraphicsPipeline :

```
VkPipelineDepthStencilStateCreateInfo depthStencil{};
depthStencil.sType                = VK_STRUCTURE_TYPE_PIPELINE_DEPTH_STENCIL_STATE_CREATE_INFO;
depthStencil.depthTestEnable      = VK_TRUE; // compare la profondeur
depthStencil.depthWriteEnable     = VK_TRUE; // ecrit dans le depth buffer si le test passe
depthStencil.depthCompareOp       = VK_COMPARE_OP_LESS; // garde le fragment le plus proche
depthStencil.depthBoundsTestEnable = VK_FALSE;
depthStencil.stencilTestEnable    = VK_FALSE;

pipelineInfo.pDepthStencilState = &depthStencil;
```

5. Ordre des appels et cleanup

initVulkan : createDepthResources doit etre appele avant createFramebuffers.

recreateSwapChain : recree les depth resources car la resolution change avec la fenetre.

cleanupSwapChain : detruire dans l'ordre :

```
vkDestroyImageView(device, depthImageView, nullptr);
vkDestroyImage(device, depthImage, nullptr);
vkFreeMemory(device, depthImageMemory, nullptr);
```

Pieges : oublier `GLM_FORCE_DEPTH_ZERO_TO_ONE` avec GLM — la matrice de projection genere des valeurs entre -1 et 1 (convention OpenGL) au lieu de 0 et 1 (Vulkan), ce qui casse le depth test. Sans depth buffer, l'ordre de rendu depend de l'ordre dans le vertex buffer, pas de la profondeur reelle.